Exercise 8 - 15.12.2025/16.12.2025

Please push all your results to GitLab!

Learning Goals

We want to practice creating a login component as an example of a reusable component which could be shared via a (library) module. Reusable components need to follow a high security standard. Therefore it is import writing tests for them in order to reuse them as a blackbox. A login form could also be a composite component example, hiding its usage of Angular Material.

We also want to get familiar with the reactive forms module in Angular.

Writing component template tests, we could test also aspects of the browser, like user events being fired or accessing state stored in the local storage. This makes them already more complex than unit tests only testing a single Typescript file. In our case with reactive forms, we can ignore most of that complexity as we can directly test the form control values based on their state (and changes) in the typescript class.

Testing full browser interaction including the real frontend-backend communication is tested in E2E tests which we will practice in exercise class 9.

Practical Exercises

1) Reuse your existing **Angular app**. Create an Angular **LoginFormComponent** (via CLI) which imports the **ReactiveFormsModule** and creates a FormGroup loginForm. The loginForm shall be a group of three input fields: **an email address, a password and a rememberMe checkbox** (boolean). Use the component constructor to initialize the form with empty / false values as default. Add a form with two inputs, a checkbox and a submit button to the template and connect it to the state in the ts file. Add **Validators.required** for email and password and **Validators.email** to verify the email address in the input field has the right format. Disable the submit button if the

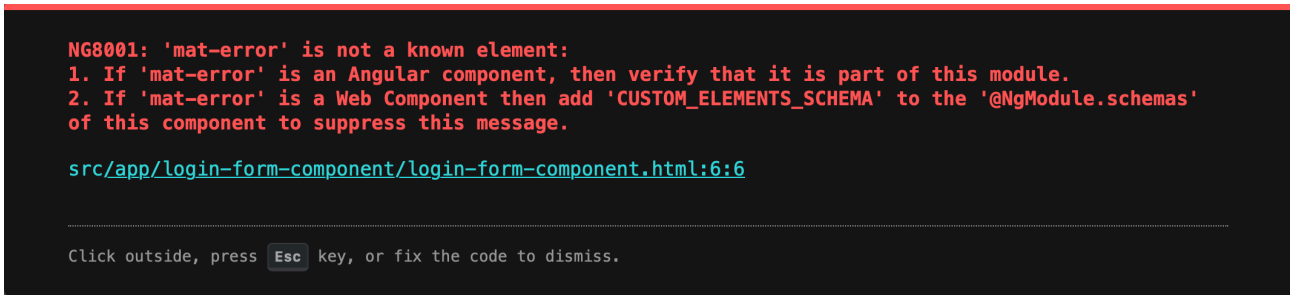
loginForm is invalid. Add a route /login to your Angular app to be able to see your component.

You can start creating the component without Material UI, we will add this in exercise 2) today. Neglecting this complexity for now can be beneficial.

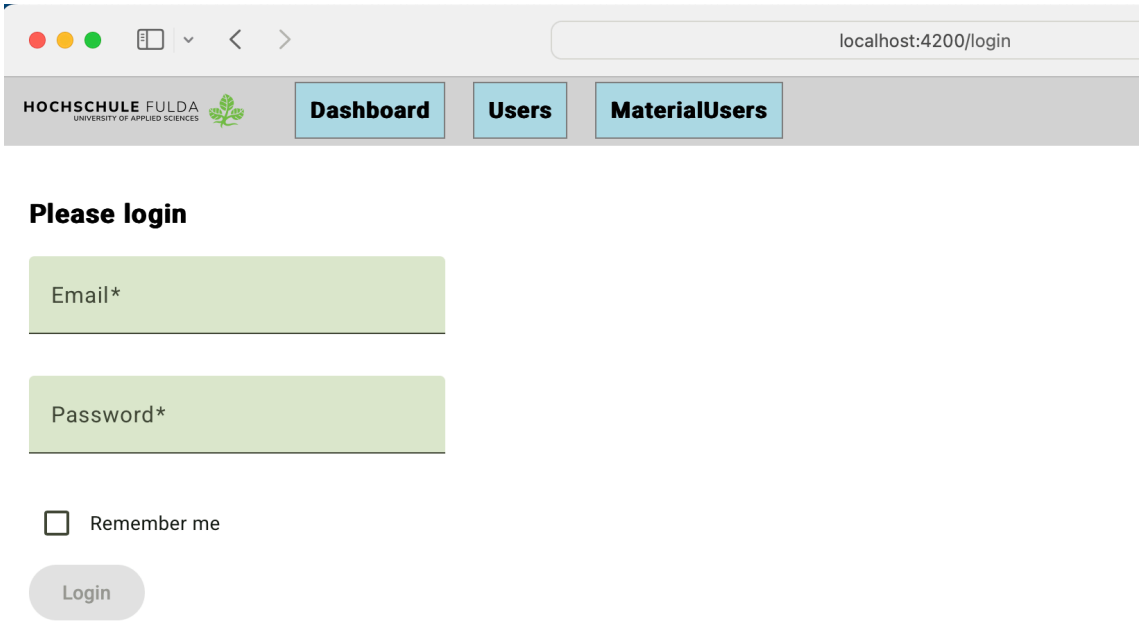
2) Use **Angular Material** to add <mat-form-field> wrapper around labels and input. Use <mat-label> for label and the matInput directive for inputs. Add <mat-error> elements to show conditionally an error message if input fields are invalid for email or password. Replace the checkbox with <mat-checkbox>.

https://material.angular.io/components/form-field/overview

You will need imports for the Checkbox and the Error as otherwise you see something like this:



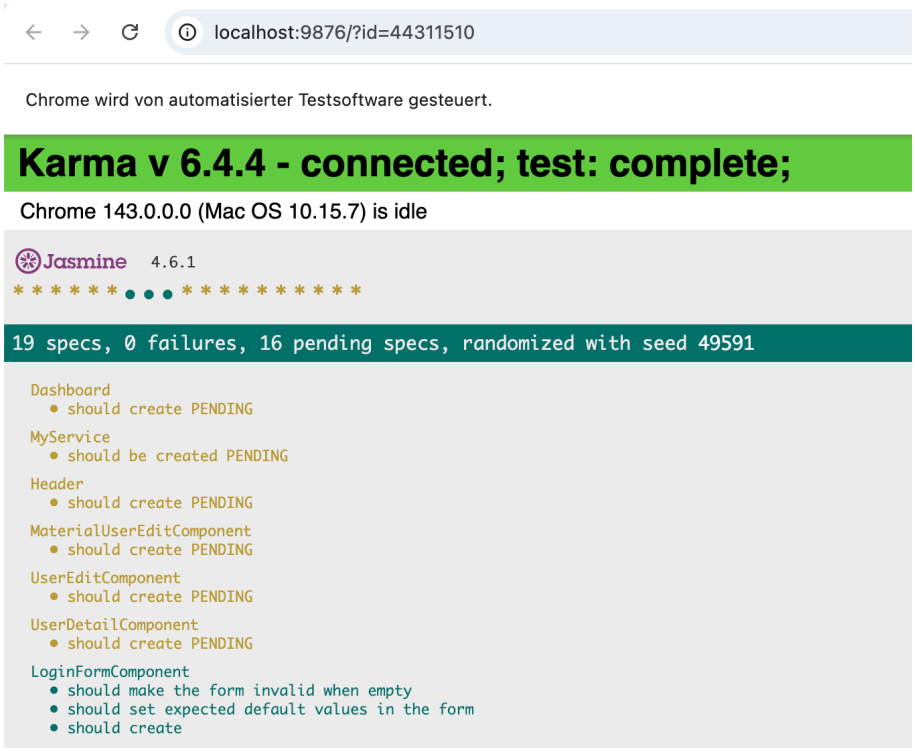
Result should look similar to this:



You should be able to type in a text which is not a valid email format and see the error message and highlighting if you click outside the input field. Only if all requirements are met, the button shall be enabled.

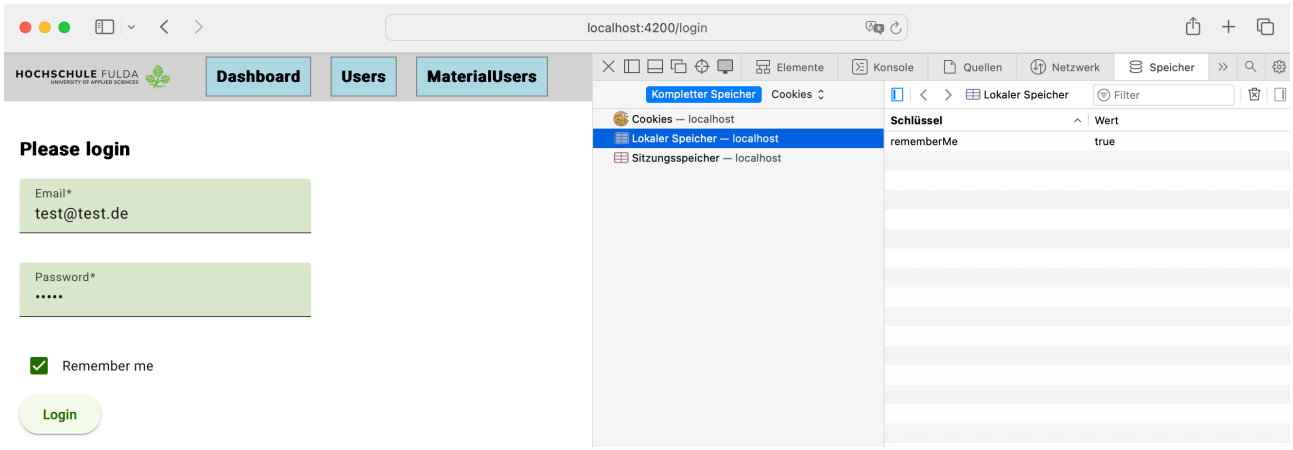
3) Add a **Jasmine test** in the already created **login-form.spec.ts file**. We want to test that initially, the loginForm.valid returns false. We also want to test that the default values (= state) in the loginForm are empty strings for the email and password.
   To ensure the component compiles correctly with all dependencies, add all the **imports** to the TestBed TestingModule which are imported in the (standalone) Component or the AppModule as well.
   You can run tests pressing play in the file in IDEA or via ***ng test***. You can ignore the execution of all other (partially broken) tests by changing *describe* to *xdescribe* for other spec files. If you have non compiling tests, just comment them out.

You should see (chrome) browser opening and the Karma test runner results being displayed:



4) Write another unit test to set different state values for the email and password loginForm.controls. Check that validity of the loginForm changes to valid = true if all criteria are met.
   You would also test if one requirements is violated (one field is invalid), that the form remains invalid. Best practice is writing a separate test case (it()) for that.

5) The checkbox „rememberMe“ shall store a boolean value to the local storage on form submit. Implement this behaviour and inspect the local storage in the browser.



6) On form submit, navigate now to /dashboard. Hint: you will need to add the „Router“ of @angular/router to the constructor of the component. What is the command to perform the navigation in the typescript code?

7) Now write a test for the onSubmit callback function of your component after having filled out the form.
   Create a spy on the global localStorage object (with its API method) and ensure the rememberMe field is set to ‚true‘ (if checked) on form submission.